

Process Scheduling

Process scheduling is the task done by the operating system to decide **which process gets the CPU next**. When one process is using the CPU, and another is waiting, the **process manager** may stop the current process and give the CPU to another, based on a specific rule or **scheduling strategy**.

One of the key responsibilities of an Operating System (OS) is to decide which programs will execute on the CPU.

There are two categories of scheduling:

1. Preemptive Scheduling

In this method, the CPU is given to a process only for a limited time.

If a higher-priority process arrives, the running process is paused, and the CPU is given to the more important one.

e.g. Imagine your computer is running two programs at the same time:

Program A: A video player (currently running and using the CPU)

Program B: An antivirus scan (starts while Program A is running)

Now, suppose the antivirus scanner (Program B) has a **higher priority** because it's a system task.

The **process manager (scheduler)** may decide:

"Stop Program A for now and give the CPU to Program B."

So, the **CPU switches** from Program A to Program B **before Program A finishes**, based on a **preemptive scheduling rule**.

e.g. Imagine you're working on your computer and doing two tasks:

Word File: You're editing your project in **Word** (MyProject.docx).

PDF File: You open a large textbook in **PDF reader** (Ebook.pdf) and scroll through pages.

Now, while you are typing in the Word document, you quickly switch to the PDF to scroll or search a keyword.

At that moment:

- The **PDF reader** requests CPU time to load new pages or process the search.
- The **Word process is still running** but not actively typing.

The **scheduler** in the operating system checks both:

- Since PDF is now the **active process** (foreground), or it needs the CPU for a short task (like rendering pages),
- The OS **pauses Word temporarily** and **gives the CPU to the PDF reader**.

After the PDF loads, the CPU may return to Word.

It gives the **illusion that both apps are working at the same time**, but actually the CPU is **switching rapidly** between them.

2. Non-Preemptive Scheduling

In this method, once a process starts running, it continues until it finishes or moves to a waiting state.

The CPU is not taken away during its execution.

e.g. ECG Device (Electrocardiogram Device)

Suppose your ECG firmware has three tasks:

Task A – Read heart sensor

Task B – Filter ECG signal

Task C – Display waveform

In non-preemptive scheduling, the CPU:

Runs Task A completely, then it stops using the CPU.

Then Task B runs, processes the data, and it stops using the CPU.

Then Task C runs and shows it on display.

No task can interrupt another task — just like in non-preemptive scheduling